

INSPECTOR T3(Knowledge Encapsulated), **Leader:** LOZA QUISHPE STEVEN SEBASTIAN

PROJECT INSPECTION **T1:**TAP. **Leader:** CARVAJAL BENITEZ JOSUE GABRIEL

Project: Parking LOTS

GitHub: <https://github.com/MattOso182/T1-OPP-Parking>

Data Folder: data

EVIDENCE:

Ord. Artifact, evaluation/10, observations:

1. Overview

Its overview is very well structured, there is not much to say

2. SRS

It was not possible to identify functional requirements with non-functional requirements in a good way.

Features

- 2.1. Register entry and exit of people who are rotating.
- 2.2.Track parking space status (available or occupied) dynamically.
- 2.3.Assign and manage parking spot.
- 2.4.Verify user and visitor authorization for access control.
- 2.5.Search and verify vehicle or license plate information.
- 2.6.Validate and update vehicle. registration automatically.
- 2.7.Manage rentals and temporary parking assignments.
- 2.8.Generate reports and usage statistics automatically.

3. Prioritized Features

The features are very well structured, there is not much to say

4. Tasks

Some tasks did not match the commits they made.

5. Plan

Their plan was quite well structured but perhaps some activities could not be carried out in the time they specified.



Design

6. Architecture

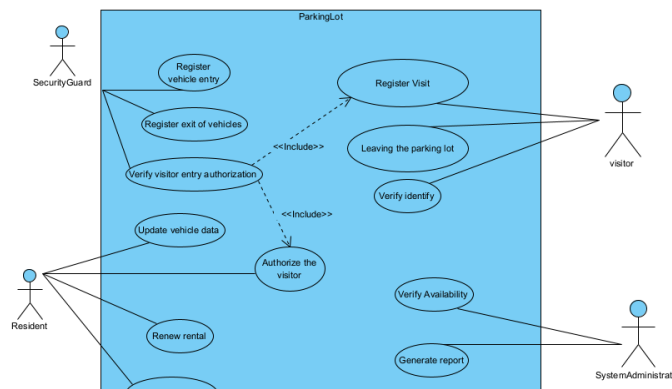
The program worked quite well, the detail that was found is that a clean and build had to be done

```

Output - ParkingControlSystem (run)
SISTEMA DE GESTION DE PARQUEADERO
=====
1. Registrar ingreso/salida (Residentes/Visitantes)
2. Rastrear estado de espacios (Ocupacion)
3. Asignar y gestionar espacios de parqueo
4. Verificar autorizacion de usuarios/visitantes
5. Buscar y verificar vehiculo/placa
6. Validar y actualizar vehiculos de residentes
7. Gestionar alquileres y asignaciones temporales
8. Generar reportes y estadisticas
9. Salir
-----
Seleccione una opcion: 4
--- [4] Verificar Autorizacion (Visitantes) ---
  
```

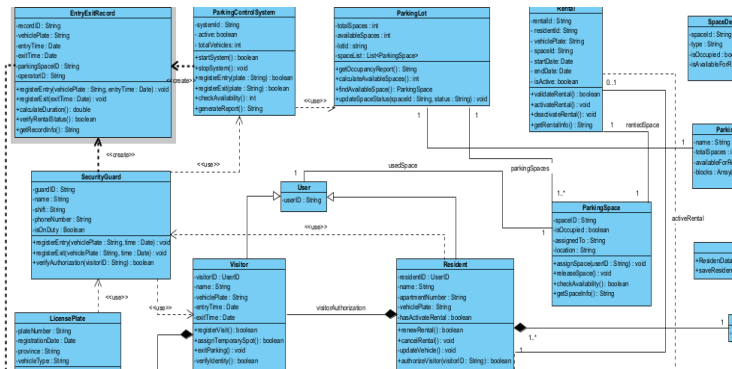
7. Use Case

Quite well structured, there is not much to say, perhaps it is too simple



8. Class Diagram

It's well done but it has a lot to cover and sometimes it's a little confusing to understand.



9. Mockups

Prototype:

CMD,

10. functionality validation

Works pretty well

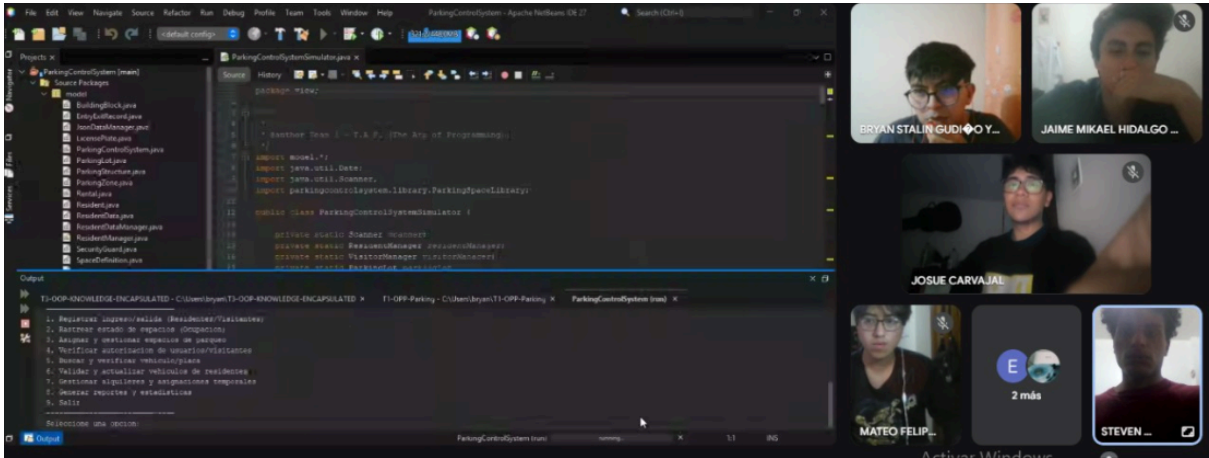
Order / Artifact		Evaluation/10	Observations
1. Overview		10	OK
2. SRS		8	functional requirements alongside non-functional requirements
Features	3. Prioritized Features	1. ok 2. ok 3. ok 4. ok 5. review the data 6. ok 7. ok 8. ok	correct part of records.
	4. Tasks	8.5	Some tasks do not match the commits in their repository
	5. Plan	9	ok
Design	6. Architecture	8.5	It works well, but to start, we had to do a clean and build in order for it to compile.
	7. Use Case	9	ok
	8. Class Diagram	Class: 8.5 Atributts: 9 Methods: 8.5	fulfills attributes and methods
	9. Mockups	8.5	The system requires the common feature which is security

Prototype: cloud	10. Functionality validation	10	ok
---------------------	---------------------------------	----	----

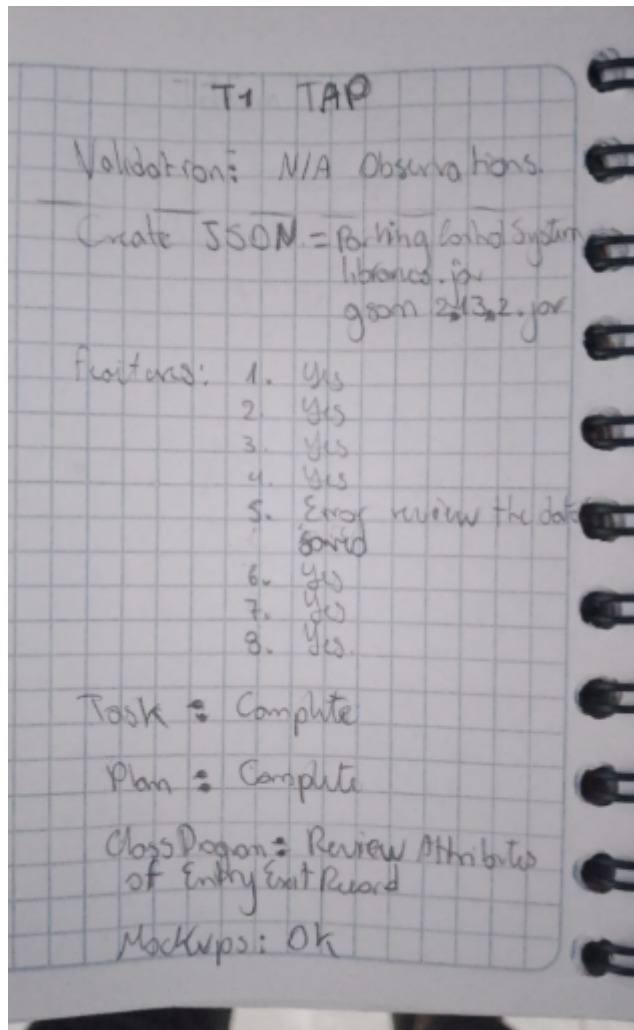
MEET EVIDENCE:



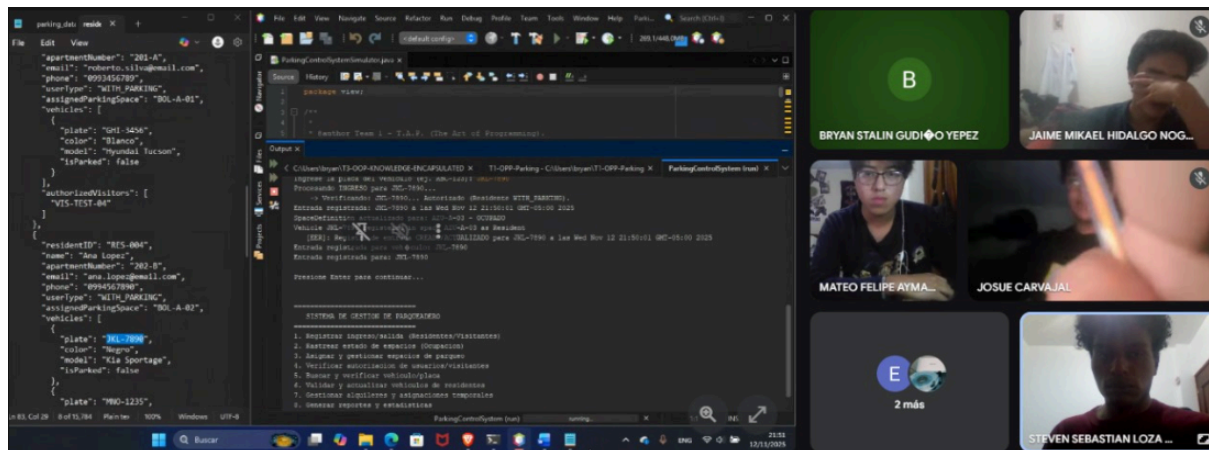
EVIDENCE CODE:



EVIDENCE OF INTERVIEW



EVIDENCE .JSON:



EVIDENCE OF COMMITS:

